

Part III

Analysis and Verification

This part of this text studies [analysis](#) of embedded systems, with emphasis on methods for specifying desired and undesired behaviors and verifying that an implementation conforms to its specification. Chapter [13](#) covers temporal logic, a formal notation that can express families of input/output behaviors and the evolution of the state of a system over time. This notation can be used to specify unambiguously desired and undesired behaviors. Chapter [14](#) explains what it means for one specification to be equivalent to another, and what it means for a design to implement a specification. Chapter [15](#) shows how to check algorithmically whether a design correctly implements a specification. Chapter [16](#) illustrates how to analyze designs for quantitative properties, with emphasis on execution time analysis for software. Such analysis is essential to achieving real-time behavior in software. Chapter [17](#) introduces the basics of security and privacy with a focus on concepts relevant to embedded, cyber-physical systems.

Invariants and Temporal Logic

13.1 Invariants	355
13.2 Linear Temporal Logic	358
13.2.1 Propositional Logic Formulas	358
13.2.2 LTL Formulas	360
<i>Sidebar: Probing Further: Alternative Temporal Logics</i>	363
13.2.3 Using LTL Formulas	365
13.3 Summary	366
<i>Sidebar: Safety and Liveness Properties</i>	367
Exercises	368

Every embedded system must be designed to meet certain requirements. Such system requirements are also called **properties** or **specifications**. The need for specifications is aptly captured by the following quotation (paraphrased from [Young et al. \(1985\)](#)):

“A design without specifications cannot be right or wrong, it can only be surprising!”

In present engineering practice, it is common to have system requirements stated in a natural language such as English. As an example, consider the SpaceWire communication

protocol that is gaining adoption with several national space agencies ([European Cooperation for Space Standardization, 2002](#)). Here are two properties reproduced from Section 8.5.2.2 of the specification document, stating conditions on the behavior of the system upon reset:

1. “The *ErrorReset* state shall be entered after a system reset, after link operation has been terminated for any reason or if there is an error during link initialization.”
2. “Whenever the reset signal is asserted the state machine shall move immediately to the *ErrorReset* state and remain there until the reset signal is de-asserted.”

It is important to precisely state requirements to avoid ambiguities inherent in natural languages. For example, consider the first property of the SpaceWire protocol stated above. Observe that there is no mention of *when* the *ErrorReset* state is to be entered. The systems that implement the SpaceWire protocol are synchronous, meaning that transitions of the state machine occur on ticks of a system clock. Given this, must the *ErrorReset* state be entered on the very next tick after one of the three conditions becomes true or on some subsequent tick of the clock? As it turns out, the document intends the system to make the transition to *ErrorReset* on the very next tick, but this is not made precise by the English language description.

This chapter will introduce techniques to specify system properties mathematically and precisely. A mathematical specification of system properties is also known as a **formal specification**. The specific formalism we will use is called **temporal logic**. As the name suggests, temporal logic is a precise mathematical notation with associated rules for representing and reasoning about timing-related properties of systems. While temporal logic has been used by philosophers and logicians since the times of Aristotle, it is only in the last thirty years that it has found application as a mathematical notation for specifying system requirements.

One of the most common kinds of system property is an **invariant**. It is also one of the simplest forms of a temporal logic property. We will first introduce the notion of an invariant and then generalize it to more expressive specifications in temporal logic.

13.1 Invariants

An **invariant** is a property that holds for a system if it remains true at all times during operation of the system. Put another way, an invariant holds for a system if it is true in the

initial state of the system, and it remains true as the system evolves, after every reaction, in every state.

In practice, many properties are invariants. Both properties of the SpaceWire protocol stated above are invariants, although this might not be immediately obvious. Both SpaceWire properties specify conditions that must remain true always. Below is an example of an invariant property of a model that we have encountered in Chapter 3.

Example 13.1: Consider the model of a traffic light controller given in Figure 3.10 and its environment as modeled in Figure 3.11. Consider the system formed by the asynchronous composition of these two state machines. An obvious property that the composed system must satisfy is that *there is no pedestrian crossing when the traffic light is green* (when cars are allowed to move). This property must always remain true of this system, and hence is a system invariant.

It is also desirable to specify invariant properties of software and hardware *implementations* of embedded systems. Some of these properties specify correct programming practice on language constructs. For example, the C language property

“The program never dereferences a null pointer”

is an invariant specifying good programming practice. Typically dereferencing a null pointer in a C program results in a [segmentation fault](#), possibly leading to a system crash. Similarly, several desirable properties of concurrent programs are invariants, as illustrated in the following example.

Example 13.2: Consider the following property regarding an absence of deadlock:

If a thread *A* blocks while trying to acquire a [mutex](#) lock, then the thread *B* that holds that lock must not be blocked attempting to acquire a lock held by *A*.

This property is required to be an invariant on any multithreaded program constructed from threads *A* and *B*. The property may or may not hold for a particular program. If it does not hold, there is risk of deadlock.

Many system invariants also impose requirements on program data, as illustrated in the example below.

Example 13.3: Consider the following example of a software task from the open source Paparazzi unmanned aerial vehicle (UAV) project (Nemer et al., 2006):

```

1 void altitude_control_task(void) {
2   if (pprz_mode == PPRZ_MODE_AUTO2
3       || pprz_mode == PPRZ_MODE_HOME) {
4     if (vertical_mode == VERTICAL_MODE_AUTO_ALT) {
5       float err = estimator_z - desired_altitude;
6       desired_climb
7         = pre_climb + altitude_pgain * err;
8       if (desired_climb < -CLIMB_MAX) {
9         desired_climb = -CLIMB_MAX;
10      }
11      if (desired_climb > CLIMB_MAX) {
12        desired_climb = CLIMB_MAX;
13      }
14    }
15  }
16 }

```

For this example, it is required that the value of the `desired_climb` variable at the end of `altitude_control_task` remains within the range `[-CLIMB_MAX, CLIMB_MAX]`. This is an example of a special kind of invariant, a **postcondition**, that must be maintained every time `altitude_control_task` returns. Determining whether this is the case requires analyzing the control flow of the program.

13.2 Linear Temporal Logic

We now give a formal description of **temporal logic** and illustrate with examples of how it can be used to specify system behavior. In particular, we study a particular kind of temporal logic known as **linear temporal logic**, or **LTL**. There are other forms of temporal logic, some of which are briefly surveyed in sidebars.

Using LTL, one can express a property over a *single, but arbitrary execution* of a system. For instance, one can express the following kinds of properties in LTL:

- *Occurrence of an event and its properties.* For example, one can express the property that an event A must occur at least once in every trace of the system, or that it must occur infinitely many times.
- *Causal dependency between events.* An example of this is the property that if an event A occurs in a trace, then event B must also occur.
- *Ordering of events.* An example of this kind of property is one specifying that every occurrence of event A is preceded by a matching occurrence of B .

We now formalize the above intuition about the kinds of properties expressible in linear temporal logic. In order to perform this formalization, it is helpful to fix a particular formal model of computation. We will use the theory of **finite-state machines**, introduced in Chapter 3.

Recall from Section 3.6 that an **execution trace** of a finite-state machine is a sequence of the form

$$q_0, q_1, q_2, q_3, \dots,$$

where $q_j = (x_j, s_j, y_j)$, s_j is the state, x_j is the input **valuation**, and y_j is the output valuation at reaction j .

13.2.1 Propositional Logic Formulas

First, we need to be able to talk about conditions at each reaction, such as whether an input or output is present, what the value of an input or output is, or what the state is. Let an **atomic proposition** be such a statement about the inputs, outputs, or states. It is a predicate (an expression that evaluates to true or false). Examples of atomic propositions that are relevant for the state machines in Figure 13.1 are:

<i>true</i>	Always true.
<i>false</i>	Always false.
<i>x</i>	True if input <i>x</i> is <i>present</i> .
<i>x = present</i>	True if input <i>x</i> is <i>present</i> .
<i>y = absent</i>	True if <i>y</i> is <i>absent</i> .
<i>b</i>	True if the FSM is in state <i>b</i>

In each case, the expression is true or false at a reaction q_i . The proposition *b* is true at a reaction q_i if $q_i = (x, b, y)$ for any valuations x and y , which means that the machine is in state *b* at the *start* of the reaction. I.e., it refers to the current state, not the next state.

A **propositional logic formula** or (more simply) **proposition** is a predicate that combines atomic propositions using **logical connectives**: **conjunction** (logical AND, denoted $\wedge$), **disjunction** (logical OR, denoted $\vee$), **negation** (logical NOT, denoted $\neg$), and **implies** (logical implication, denoted $\implies$). Propositions for the state machines in Figure 13.1 include any of the above atomic proposition and expressions using the logical connectives together with atomic propositions. Here are some examples:

$x \wedge y$	True if x and y are both <i>present</i> .
$x \vee y$	True if either x or y is <i>present</i> .
$x = \text{present} \wedge y = \text{absent}$	True if x is <i>present</i> and y is <i>absent</i> .
$\neg y$	True if y is <i>absent</i> .
$a \implies y$	True if whenever the FSM is in state <i>a</i> , the output y will be made <i>present</i> by the reaction

input: x : pure
output: y : pure

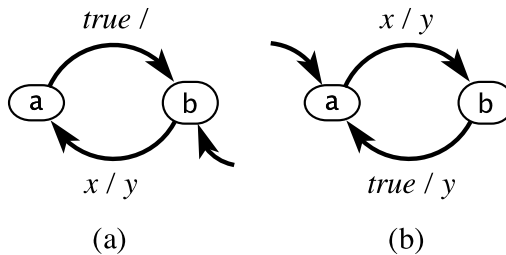


Figure 13.1: Two finite-state machines used to illustrate LTL formulas.

Note that if p_1 and p_2 are propositions, the proposition $p_1 \implies p_2$ is true if and only if $\neg p_2 \implies \neg p_1$. In other words, if we wish to establish that $p_1 \implies p_2$ is true, it is equally valid to establish that $\neg p_2 \implies \neg p_1$ is true. In logic, the latter expression is called the **contrapositive** of the former.

Note further that $p_1 \implies p_2$ is true if p_1 is false. This is easy to see by considering the contrapositive. The proposition $\neg p_2 \implies \neg p_1$ is true regardless of p_2 if $\neg p_1$ is true. Thus, another proposition that is equivalent to $p_1 \implies p_2$ is

$$\neg p_1 \vee p_2 .$$

13.2.2 LTL Formulas

An **LTL formula**, unlike the above propositions, applies to an entire trace

$$q_0, q_1, q_2, \dots,$$

rather than to just one reaction q_i . The simplest LTL formulas look just like the propositions above, but they apply to an entire trace rather than just a single element of the trace. If p is a proposition, then by definition, we say that LTL formula $\phi = p$ **holds** for the trace $q_0, q_1, q_2, \dots$ if and only if p is true for q_0 . It may seem odd to say that the formula holds for the entire trace even though the proposition only holds for the first element of the trace, but we will see that LTL provides ways to reason about the entire trace.

By convention, we will denote LTL formulas by ϕ, ϕ_1, ϕ_2 , etc. and propositions by p, p_1, p_2 , etc.

Given a state machine M and an LTL formula ϕ , we say that ϕ holds for M if ϕ holds for all possible traces of M . This typically requires considering all possible inputs.

Example 13.4: The LTL formula a holds for Figure 13.1(b), because all traces begin in state a . It does not hold for Figure 13.1(a).

The LTL formula $x \implies y$ holds for both machines. In both cases, in the first reaction, if x is *present*, then y will be *present*.

To demonstrate that an LTL formula is false for an FSM, it is sufficient to give one trace for which it is false. Such a trace is called a **counterexample**. To show that an LTL formula is true for an FSM, you must demonstrate that it is true for all traces, which is often much harder (although not so much harder when the LTL formula is a simple propositional logic formula, because in that case we only have to consider the first element of the trace).

Example 13.5: The LTL formula y is false for both FSMs in Figure 13.1. In both cases, a counterexample is a trace where x is absent in the first reaction.

In addition to propositions, LTL formulas can also have one or more special **temporal operators**. These make LTL much more interesting, because they enable reasoning about entire traces instead of just making assertions about the first element of a trace. There are four main temporal operators, which we describe next.

G Operator

The property $\mathbf{G}\phi$ (which is read as “**globally** ϕ ”) holds for a trace if ϕ holds for *every* suffix of that trace. (A **suffix** is a tail of a trace beginning with some reaction and including all subsequent reactions.)

In mathematical notation, $\mathbf{G}\phi$ holds for the trace if and only if, for *all* $j \geq 0$, formula ϕ holds in the suffix $q_j, q_{j+1}, q_{j+2}, \dots$

Example 13.6: In Figure 13.1(b), $\mathbf{G}(x \implies y)$ is true for all traces of the machine, and hence holds for the machine. $\mathbf{G}(x \wedge y)$ does not hold for the machine, because it is false for any trace where x is absent in any reaction. Such a trace provides a counterexample.

If ϕ is a propositional logic formula, then $\mathbf{G}\phi$ simply means that ϕ holds in every reaction. We will see, however, that when we combine the $\mathbf{G}$ operator with other temporal logic

operators, we can make much more interesting statements about traces and about state machines.

F Operator

The property $\mathbf{F}\phi$ (which is read as “**eventually** ϕ ” or “**finally** ϕ ”) holds for a trace if ϕ holds for *some* suffix of the trace.

Formally, $\mathbf{F}\phi$ holds for the trace if and only if, for *some* $j \geq 0$, formula ϕ holds in the suffix $q_j, q_{j+1}, q_{j+2}, \dots$

Example 13.7: In Figure 13.1(a), $\mathbf{Fb}$ is trivially true because the machine starts in state b, hence, for all traces, the proposition b holds for the trace itself (the very first suffix).

More interestingly, $\mathbf{G}(x \implies \mathbf{Fb})$ holds for Figure 13.1(a). This is because if x is *present* in any reaction, then the machine will eventually be in state b. This is true even in suffixes that start in state a.

Notice that parentheses can be important in interpreting an LTL formula. For example, $(\mathbf{G}x) \implies (\mathbf{Fb})$ is trivially true because $\mathbf{Fb}$ is true for all traces (since the initial state is b).

Notice that $\mathbf{F}\neg\phi$ holds if and only if $\neg\mathbf{G}\phi$. That is, stating that ϕ is eventually false is the same as stating that ϕ is not always true.

X Operator

The property $\mathbf{X}\phi$ (which is read as “**next state** ϕ ”) holds for a trace $q_0, q_1, q_2, \dots$ if and only if ϕ holds for the trace $q_1, q_2, q_3, \dots$

Example 13.8: In Figure 13.1(a), $x \implies \mathbf{Xa}$ holds for the state machine, because if x is *present* in the first reaction, then the next state will be a. $\mathbf{G}(x \implies$

Probing Further: Alternative Temporal Logics

Amir Pnueli (1977) was the first to formalize temporal logic as a way of specifying program properties. For this he won the 1996 ACM Turing Award, the highest honor in Computer Science. Since his seminal paper, temporal logic has become widespread as a way of specifying properties for a range of systems, including hardware, software, and cyber-physical systems.

In this chapter, we have focused on **LTL**, but there are several alternatives. LTL formulas apply to individual traces of an FSM, and in this chapter, by convention, we assert that an LTL formula holds for an FSM if it holds for all possible traces of the FSM. A more general logic called **computation tree logic (CTL*)** explicitly provides quantifiers over possible traces of an FSM (Emerson and Clarke (1980); Ben-Ari et al. (1981)). For example, we can write a CTL* expression that holds for an FSM if there exists *any* trace that satisfies some property, rather than insisting that the property must hold *for all* traces. CTL* is called a **branching-time logic** because whenever a reaction of the FSM has a nondeterministic choice, it will simultaneously consider all options. LTL, by contrast, considers only one trace at a time, and hence it is called a **linear-time logic**. Our convention of asserting that an LTL formula holds for an FSM if it holds for all traces cannot be expressed directly in LTL, because LTL does not include quantifiers like “for all traces.” We have to step outside the logic to apply this convention. With CTL*, this convention is expressible directly in the logic.

Several other temporal logic variants have found practical use. For instance, **real-time temporal logics** (e.g., **timed computation tree logic** or **TCTL**), is used for reasoning about **real-time** systems (Alur et al., 1991; Alur and Henzinger, 1993) where the passage of time is not in discrete steps, but is continuous. Similarly, **probabilistic temporal logics** are useful for reasoning about probabilistic models such as Markov chains or Markov decision processes (see, for example, Hansson and Jonsson (1994)), and **signal temporal logic** has proved effective for reasoning about real-time behavior of **hybrid systems** (Maler and Nickovic, 2004).

Techniques for inferring temporal logic properties from traces, also known as **specification mining**, have also proved useful in industrial practice (see Jin et al. (2015)).

$\mathbf{X}a$) does not hold for the state machine because it does not hold for any suffix that begins in state a . In Figure 13.1(b), $\mathbf{G}(b \implies \mathbf{X}a)$ holds for the state machine.

U Operator

The property $\phi_1 \mathbf{U} \phi_2$ (which is read as “ ϕ_1 **until** ϕ_2 ”) holds for a trace if ϕ_2 holds for some suffix of that trace, and ϕ_1 holds until ϕ_2 becomes *true*.

Formally, $\phi_1 \mathbf{U} \phi_2$ holds for the trace if and only if there exists $j \geq 0$ such that ϕ_2 holds in the suffix $q_j, q_{j+1}, q_{j+2}, \dots$ and ϕ_1 holds in suffixes $q_i, q_{i+1}, q_{i+2}, \dots$, for all i s.t. $0 \leq i < j$. ϕ_1 may or may not hold for $q_j, q_{j+1}, q_{j+2}, \dots$

Example 13.9: In Figure 13.1(b), $a\mathbf{U}x$ is true for any trace for which $\mathbf{F}x$ holds. Since this does not include all traces, $a\mathbf{U}x$ does not hold for the state machine.

Some authors define a weaker form of the $\mathbf{U}$ operator that does not require ϕ_2 to hold. Using our definition, this can be written

$$(\mathbf{G}\phi_1) \vee (\phi_1 \mathbf{U} \phi_2) .$$

This holds if either ϕ_1 always holds (for any suffix) or, if ϕ_2 holds for some suffix, then ϕ_1 holds for all previous suffixes. This can equivalently be written

$$(\mathbf{F}\neg\phi_1) \implies (\phi_1 \mathbf{U} \phi_2) .$$

Example 13.10: In Figure 13.1(b), $(\mathbf{G}\neg x) \vee (a\mathbf{U}x)$ holds for the state machine.

13.2.3 Using LTL Formulas

Consider the following English descriptions of properties and their corresponding LTL formalizations:

Example 13.11: *“Whenever the robot is facing an obstacle, eventually it moves at least 5 cm away from the obstacle.”*

Let p denote the condition that the robot is facing an obstacle, and q denote the condition where the robot is at least 5 cm away from the obstacle. Then, this property can be formalized in LTL as

$$\mathbf{G} (p \implies \mathbf{F} q) .$$

Example 13.12: Consider the SpaceWire property:

“Whenever the reset signal is asserted the state machine shall move immediately to the ErrorReset state and remain there until the reset signal is de-asserted.”

Let p be *true* when the reset signal is asserted, and q be true when the state of the FSM is *ErrorReset*. Then, the above English property is formalized in LTL as:

$$\mathbf{G} (p \implies \mathbf{X}(q \mathbf{U} \neg p)) .$$

In the above formalization, we have interpreted “immediately” to mean that the state changes to *ErrorReset* in the very next time step. Moreover, the above LTL formula will fail to hold for any execution where the reset signal is asserted and not eventually de-asserted. It was probably the intent of the standard that the reset signal should be eventually de-asserted, but the English language statement does not make this clear.

Example 13.13: Consider the traffic light controller in Figure 3.10. A property of this controller is that the outputs always cycle through sigG , sigY and sigR ,

unless no pedestrian input appears and the machine stays in the green state. We can express this in LTL as follows:

$$\begin{aligned} \mathbf{G} \{ & (sigG \implies \mathbf{X}(\mathbf{G}(\neg sigR \wedge \neg sigG) \vee (\neg sigR \wedge \neg sigG) \mathbf{U} sigY)) \\ & \wedge (sigY \implies \mathbf{X}((\neg sigG \wedge \neg sigY) \mathbf{U} sigR)) \\ & \wedge (sigR \implies \mathbf{X}((\neg sigY \wedge \neg sigR) \mathbf{U} sigG)) \} . \end{aligned}$$

The following LTL formulas express commonly useful properties.

- (a) *Infinitely many occurrences*: This property is of the form $\mathbf{G} \mathbf{F}p$, meaning that it is always the case that p is *true* eventually. Put another way, this means that p is true **infinitely often**.
- (b) *Steady-state property*: This property is of the form $\mathbf{F} \mathbf{G}p$, read as “from some point in the future, p holds at all times.” This represents a steady-state property, indicating that after some point in time, the system reaches a **steady state** in which p is always *true*.
- (c) *Request-response property*: The formula $\mathbf{G}(p \implies \mathbf{F}q)$ can be interpreted to mean that a request p will eventually produce a response q .

13.3 Summary

Dependability and correctness are central concerns in embedded systems design. Formal specifications, in turn, are central to achieving these goals. In this chapter, we have studied temporal logic, one of the main approaches for writing formal specifications. This chapter has provided techniques for precisely stating properties that must hold over time for a system. It has specifically focused on linear temporal logic, which is able to express many safety and liveness properties of systems.

Safety and Liveness Properties

System properties may be **safety** or **liveness** properties. Informally, a safety property is one specifying that “nothing bad happens” during execution. Similarly, a liveness property specifies that “something good will happen” during execution.

More formally, a property p is a **safety property** if a system execution does not satisfy p if and only if there exists a finite-length prefix of the execution that cannot be extended to an infinite execution satisfying p . We say p is a **liveness property** if every finite-length execution trace can be extended to an infinite execution that satisfies p . See [Lamport \(1977\)](#) and [Alpern and Schneider \(1987\)](#) for a theoretical treatment of safety and liveness.

The properties we have seen in Section 13.1 are all examples of safety properties. Liveness properties, on the other hand, specify performance or progress requirements on a system. For a state machine, a property of the form $\mathbf{F}\phi$ is a liveness property. No finite execution can establish that this property is not satisfied.

The following is a slightly more elaborate example of a liveness property:

“Whenever an interrupt is asserted, the corresponding interrupt service routine (ISR) is eventually executed.”

In temporal logic, if p_1 is the property that an interrupt is asserted, and p_2 is the property that the interrupt service routine is executed, then this property can be written

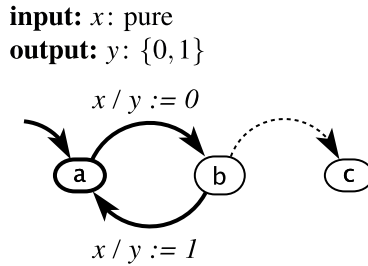
$$\mathbf{G}(p_1 \implies \mathbf{F}p_2) .$$

Note that both safety and liveness properties can constitute system invariants. For example, the above liveness property on interrupts is also an invariant; $p_1 \implies \mathbf{F}p_2$ must hold in *every state*.

Liveness properties can be either *bounded* or *unbounded*. A **bounded liveness** property specifies a time bound on something desirable happening (which makes it a safety property). In the above example, if the ISR must be executed within 100 clock cycles of the interrupt being asserted, the property is a bounded liveness property; otherwise, if there is no such time bound on the occurrence of the ISR, it is an **unbounded liveness** property. LTL can express a limited form of bounded liveness properties using the **X** operator, but it does not provide any mechanism for quantifying time directly.

Exercises

1. For each of the following questions, give a short answer and justification.
 - (a) TRUE or FALSE: If $\mathbf{GF}p$ holds for a state machine A , then so does $\mathbf{FG}p$.
 - (b) TRUE or FALSE: $\mathbf{G}(\mathbf{G}p)$ holds for a trace if and only if $\mathbf{G}p$ holds.
2. Consider the following state machine:



(Recall that the dashed line represents a [default transition](#).) For each of the following LTL formulas, determine whether it is true or false, and if it is false, give a counterexample:

- (a) $x \implies \mathbf{F}b$
 - (b) $\mathbf{G}(x \implies \mathbf{F}(y = 1))$
 - (c) $(\mathbf{G}x) \implies \mathbf{F}(y = 1)$
 - (d) $(\mathbf{G}x) \implies \mathbf{GF}(y = 1)$
 - (e) $\mathbf{G}((b \wedge \neg x) \implies \mathbf{FG}c)$
 - (f) $\mathbf{G}((b \wedge \neg x) \implies \mathbf{G}c)$
 - (g) $(\mathbf{GF}\neg x) \implies \mathbf{FG}c$
3. Consider the synchronous feedback composition studied in Exercise 6 of Chapter 6. Determine whether the following statement is true or false:

The following temporal logic formula is satisfied by the sequence w for every possible behavior of the composition and is not satisfied by any sequence that is not a behavior of the composition:

$$(\mathbf{G}w) \vee (w\mathbf{U}(\mathbf{G}\neg w)) .$$

Justify your answer. If you decide it is false, then provide a temporal logic formula for which the assertion is true.

4. This problem is concerned with specifying in linear temporal logic tasks to be performed by a robot. Suppose the robot must visit a set of n locations $l_1, l_2, \dots, l_n$. Let p_i be an atomic formula that is *true* if and only if the robot visits location l_i .

Give LTL formulas specifying the following tasks:

- (a) The robot must eventually visit at least one of the n locations.
- (b) The robot must eventually visit all n locations, but in any order.
- (c) The robot must eventually visit all n locations, in the order $l_1, l_2, \dots, l_n$.

5. Consider a system M modeled by the hierarchical state machine of Figure 13.2, which models an interrupt-driven program. M has two modes: *Inactive*, in which the main program executes, and *Active*, in which the interrupt service routine (ISR) executes. The main program and ISR read and update a common variable *timer-Count*.

Answer the following questions:

- (a) Specify the following property ϕ in linear temporal logic, choosing suitable atomic propositions:

ϕ : The main program eventually reaches program location C.

- (b) Does M satisfy the above LTL property? Justify your answer by constructing the product FSM. If M does not satisfy the property, under what conditions would it do so? Assume that the environment of M can assert the interrupt at any time.
6. Express the [postcondition](#) of Example 13.3 as an LTL formula. State your assumptions clearly.
7. Consider the program fragment shown in Figure 11.6, which provides procedures for threads to communicate asynchronously by sending messages to one another. Please answer the following questions about this code. Assume the code is running on a single processor (not a multicore machine). You may also assume that only the code shown accesses the static variables that are shown.

variables: *timerCount*: uint
input: *assert*: pure, *return*: pure
output: *return*: pure

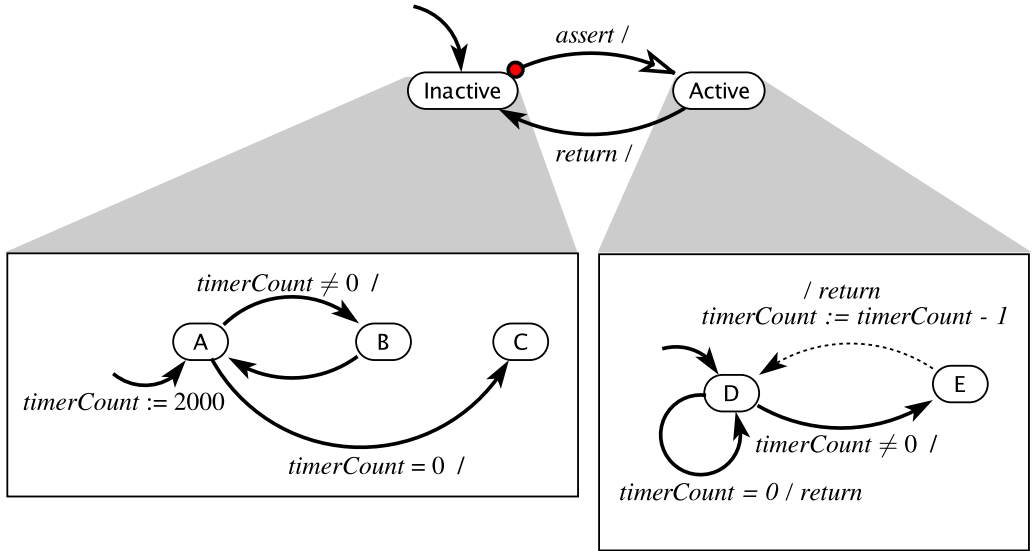


Figure 13.2: Hierarchical state machine modeling a program and its interrupt service routine.

- Let s be an atomic proposition asserting that `send` releases the mutex (i.e. executes line 24). Let g be an atomic proposition asserting that `get` releases the mutex (i.e. executes line 38). Write an LTL formula asserting that g cannot occur before s in an execution of the program. Does this formula hold for the first execution of any program that uses these procedures?
- Suppose that a program that uses the `send` and `get` procedures in Figure 11.6 is aborted at an arbitrary point in its execution and then restarted at the beginning. In the new execution, it is possible for a call to `get` to return before any call to `send` has been made. Describe how this could come about. What value will `get` return?
- Suppose again that a program that uses the `send` and `get` procedures above is aborted at an arbitrary point in its execution and then restarted at the beginning. In the new execution, is it possible for deadlock to occur, where neither

a call to `get` nor a call to `send` can return? If so, describe how this could come about and suggest a fix. If not, give an argument.

Equivalence and Refinement

14.1 Models as Specifications	373
14.2 Type Equivalence and Refinement	374
<i>Sidebar: Abstraction and Refinement</i>	374
14.3 Language Equivalence and Containment	377
<i>Sidebar: Finite Sequences and Accepting States</i>	380
<i>Sidebar: Regular Languages and Regular Expressions</i>	381
<i>Sidebar: Probing Further: Omega Regular Languages</i>	382
14.4 Simulation	383
14.4.1 Simulation Relations	385
14.4.2 Formal Model	387
14.4.3 Transitivity	388
14.4.4 Non-Uniqueness of Simulation Relations	389
14.4.5 Simulation vs. Language Containment	389
14.5 Bisimulation	391
14.6 Summary	394
Exercises	395

This chapter discusses some fundamental ways to compare state machines and other modal models, such as trace equivalence, trace containment, simulation, and bisimulation. These mechanisms can be used to check conformance of a state machine against a specification.

14.1 Models as Specifications

The previous chapter provided techniques for unambiguously stating properties that a system must have to be functioning properly and safely. These properties were expressed using [linear temporal logic](#), which can concisely describe requirements that the trace of a finite-state machine must satisfy. An alternative way to give requirements is to provide a model, a [specification](#), that exhibits expected behavior of the system. Typically, the specification is quite abstract, and it may exhibit more behaviors than a useful implementation of the system would. But the key to being a useful specification is that it explicitly excludes undesired or dangerous behaviors.

Example 14.1: A simple specification for a traffic light might state: “The lights should always be lighted in the order green, yellow, red. It should never, for example, go directly from green to red, or from yellow to green.” This requirement can be given as a temporal logic formula (as is done in [Example 13.13](#)) or as an abstract model (as is done in [Figure 3.12](#)).

The topic of this chapter is on the use of abstract models as specifications, and on how such models relate to an implementation of a system and to temporal logic formulas.

Example 14.2: We will show how to demonstrate that the traffic light model shown in [Figure 3.10](#) is a valid implementation of the specification in [Figure 3.12](#). Moreover, all traces of the model in [Figure 3.10](#) satisfy the temporal logic formula in [Example 13.13](#), but not all traces of the specification in [Figure 3.12](#) do. Hence, these two specifications are not the same.

This chapter is about comparing models, and about being able to say with confidence that one model can be used in place of another. This enables an engineering design process where we start with abstract descriptions of desired and undesired behaviors, and successively refine our models until we have something that is detailed enough to provide

a complete implementation. It also tells when it is safe to change an implementation, replacing it with another that might, for example, reduce the implementation cost.

14.2 Type Equivalence and Refinement

We begin with a simple relationship between two models that compares only the data [types](#) of their communication with their environment. Specifically, the goal is to ensure that a model B can be used in any environment where a model A can be used without causing any conflicts about data types. We will require that B can accept any inputs that

Abstraction and Refinement

This chapter focuses on relationships between models known as **abstraction** and **refinement**. These terms are symmetric in that the statement “model A is an abstraction of model B ” means the same thing as “model B is a refinement of model A .” As a general rule, the refinement model B has more detail than the abstraction A , and the abstraction is simpler, smaller, or easier to understand.

An abstraction is **sound** (with respect to some formal system of properties) if properties that are true of the abstraction are also true of the refinement. The formal system of properties could be, for example, a [type](#) system, [linear temporal logic](#), or the [languages](#) of state machines. If the formal system is LTL, then if every LTL formula that holds for A also holds for B , then A is a sound abstraction of B . This is useful when it is easier to prove that a formula holds for A than to prove that it holds for B , for example because the state space of B may be much larger than the state space of A .

An abstraction is **complete** (with respect to some formal system of properties) if properties that are true of the refinement are also true of the abstraction. For example, if the formal system of properties is LTL, then A is a complete abstraction of B if every LTL formula that holds for B also holds for A . Useful abstractions are usually sound but not complete, because it is hard to make a complete abstraction that is significantly simpler or smaller.

Consider for example a program B in an [imperative](#) language such as C that has multiple [threads](#). We might construct an abstraction A that ignores the values of variables and replaces all branches and control structures with [nondeterministic](#) choices. The abstraction clearly has less information than the program, but it may be sufficient for proving some properties about the program, for example a [mutual exclusion](#) property.

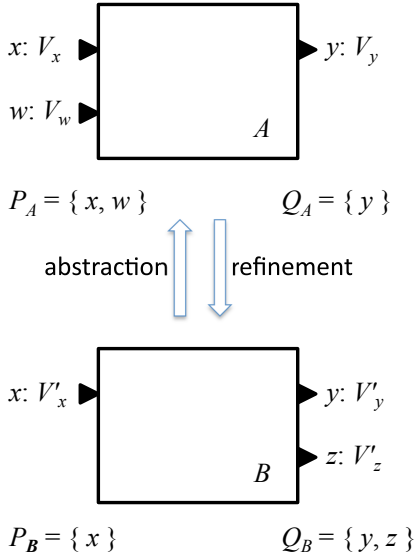
A can accept from the environment, and that any environment that can accept any output A can produce can also accept any output that B can produce.

To make the problem concrete, assume an **actor** model for A and B , as shown in Figure 14.1. In that figure, A has three **ports**, two of which are input ports represented by the set $P_A = \{x, w\}$, and one of which is an output port represented by the set $Q_A = \{y\}$. These ports represent communication between A and its environment. The inputs have type V_x and V_w , which means that at a **reaction** of the actor, the values of the inputs will be members of the sets V_x or V_w .

If we want to replace A by B in some environment, the ports and their types impose four constraints:

1. The first constraint is that B does not require some input signal that the environment does not provide. If the input ports of B are given by the set P_B , then this is guaranteed by

$$P_B \subseteq P_A. \quad (14.1)$$



- (1) $P_B \subseteq P_A$
- (2) $Q_A \subseteq Q_B$
- (3) $\forall p \in P_B, \quad V_p \subseteq V'_p$
- (4) $\forall q \in Q_A, \quad V'_q \subseteq V_q$

Figure 14.1: Summary of type refinement. If the four constraints on the right are satisfied, then B is a type refinement of A .

The ports of B are a subset of the ports of A . It is harmless for A to have more input ports than B , because if B replaces A in some environment, it can simply ignore any input signals that it does not need.

2. The second constraint is that B produces all the output signals that the environment may require. This is ensured by the constraint

$$Q_A \subseteq Q_B, \quad (14.2)$$

where Q_A is the set of output ports of A , and Q_B is the set of output ports of B . It is harmless for B to have additional output ports because an environment capable of working with A does not expect such outputs and hence can ignore them.

The remaining two constraints deal with the types of the ports. Let the type of an input port $p \in P_A$ be given by V_p . This means that an acceptable input value v on p satisfies $v \in V_p$. Let V'_p denote the type of an input port $p \in P_B$.

3. The third constraint is that if the environment provides a value $v \in V_p$ on an input port p that is acceptable to A , then if p is also an input port of B , then the value is also acceptable to B ; i.e., $v \in V'_p$. This constraint can be written compactly as follows,

$$\forall p \in P_B, \quad V_p \subseteq V'_p. \quad (14.3)$$

Let the type of an output port $q \in Q_A$ be V_q , and the type of the corresponding output port $q \in Q_B$ be V'_q .

4. The fourth constraint is that if B produces a value $v \in V'_q$ on an output port q , then if q is also an output port of A , then the value must be acceptable to any environment in which A can operate. In other words,

$$\forall q \in Q_A, \quad V'_q \subseteq V_q. \quad (14.4)$$

The four constraints of equations (14.1) through (14.4) are summarized in Figure 14.1. When these four constraints are satisfied, we say that B is a **type refinement** of A . If B is a type refinement of A , then replacing A by B in any environment will not cause type system problems. It could, of course, cause other problems, since the behavior of B may

not be acceptable to the environment, but that problem will be dealt with in subsequent sections.

If B is a type refinement of A , and A is a type refinement of B , then we say that A and B are **type equivalent**. They have the same input and output ports, and the types of the ports are the same.

Example 14.3: Let A represent the nondeterministic traffic light model in Figure 3.12 and B represent the more detailed deterministic model in Figure 3.10. The ports and their types are identical for both machines, so they are type equivalent. Hence, replacing A with B or vice versa in any environment will not cause type system problems.

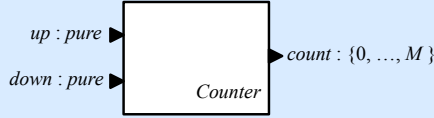
Notice that since Figure 3.12 ignores the *pedestrian* input, it might seem reasonable to omit that port. Let A' represent a variant of Figure 3.12 without the *pedestrian* input. It is not safe to replace A' with B in all environments, because B requires an input *pedestrian* signal, but A' can be used in an environment that provides no such input.

14.3 Language Equivalence and Containment

To replace a machine A with a machine B , looking at the data types of the inputs and outputs alone is usually not enough. If A is a specification and B is an implementation, then normally A imposes more constraints than just data types. If B is an optimization of A (e.g., a lower cost implementation or a refinement that adds functionality or leverages new technology), then B normally needs to conform in some way with the functionality of A .

In this section, we consider a stronger form of equivalence and refinement. Specifically, equivalence will mean that given a particular sequence of input **valuations**, the two machines produce the same output valuations.

Example 14.4: The garage counter of Figure 3.4, discussed in Example 3.4, is type equivalent to the [extended state machine](#) version in Figure 3.8. The actor model is shown below:



However, these two machines are equivalent in a much stronger sense than simply type equivalence. These two machines behave in exactly the same way, as viewed from the outside. Given the same input sequence, the two machines will produce the same output sequence.

Consider a port p of a state machine with type V_p . This port will have a sequence of values from the set $V_p \cup \{absent\}$, one value at each reaction. We can represent this sequence as a function of the form

$$s_p : \mathbb{N} \rightarrow V_p \cup \{absent\}.$$

This is the signal received on that port (if it is an input) or produced on that port (if it is an output). Recall that a [behavior](#) of a state machine is an assignment of such a signal to each port of such a machine. Recall further that the [language](#) $L(M)$ of a state machine M is the set of all behaviors for that state machine. Two machines are said to be **language equivalent** if they have the same language.

Example 14.5: A behavior of the garage counter is a sequence of *present* and *absent* valuations for the two inputs, *up* and *down*, paired with the corresponding output sequence at the output port, *count*. A specific example is given in Example 3.16. This is a behavior of both Figures 3.4 and 3.8. All behaviors of Figure 3.4 are also behaviors of 3.8 and vice versa. These two machines are language equivalent.

In the case of a [nondeterministic](#) machine M , two distinct behaviors may share the same input signals. That is, given an input signal, there is more than one possible output se-

input: x : pure
output: y : $\{0, 1\}$

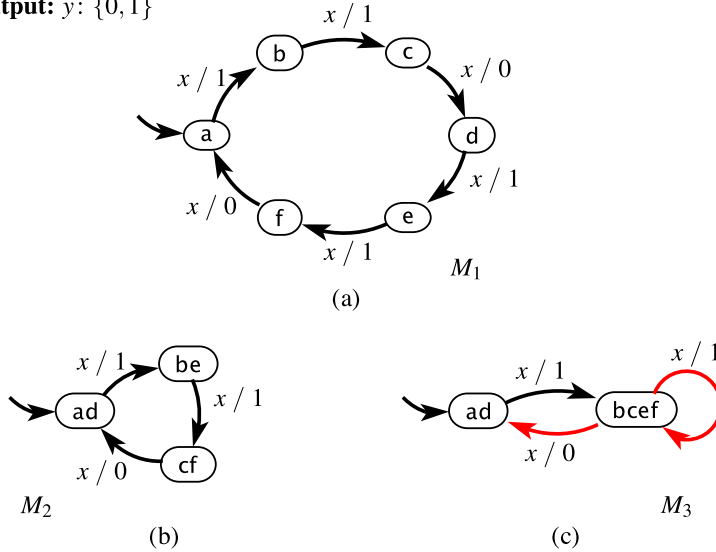


Figure 14.2: Three state machines where (a) and (b) have the same language, and that language is contained by that of (c).

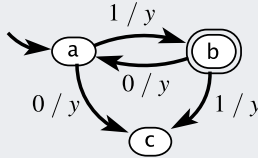
quence. The language $L(M)$ includes all possible behaviors. Just like deterministic machines, two nondeterministic machines are language equivalent if they have the same language.

Suppose that for two state machines A and B , $L(A) \subset L(B)$. That is, B has behaviors that A does not have. This is called **language containment**. A is said to be a **language refinement** of B . Just as with **type refinement**, language refinement makes an assertion about the suitability of A as a replacement for B . If every behavior of B is acceptable to an environment, then every behavior of A will also be acceptable to that environment. A can substitute for B .

Finite Sequences and Accepting States

A complete execution of the FSMs considered in this text is infinite. Suppose that we are interested in only the finite executions. To do this, we introduce the notion of an **accepting state**, indicated with a double outline as in state b in the example below:

input: $x: \{0, 1\}$
output: $y: \text{pure}$



Let $L_a(M)$ denote the subset of the language $L(M)$ that results from executions that terminate in an accepting state. Equivalently, $L_a(M)$ includes only those behaviors in $L(M)$ with an infinite tail of **stuttering** reactions that remain in an accepting state. All such executions are effectively finite, since after a finite number of reactions, the inputs and outputs will henceforth be *absent*, or in **LTL**, $\mathbf{FG}\neg p$ for every port p .

We call $L_a(M)$ the **language accepted by an FSM** M . A behavior in $L_a(M)$ specifies for each port p a finite **string**, or a finite sequence of values from the type V_p . For the above example, the input strings (1), (1, 0, 1), (1, 0, 1, 0, 1), etc., are all in $L_a(M)$. So are versions of these with an arbitrary finite number of *absent* values between any two present values. When there is no ambiguity, we can write these strings 1, 101, 10101, etc.

In the above example, in all behaviors in $L_a(M)$, the output is present a finite number of times, in the same reactions when the input is present.

The state machines in this text are **receptive**, meaning that at each reaction, each input port p can have any value in its type V_p or be *absent*. Hence, the language $L(M)$ of the machine above includes all possible sequences of input valuations. $L_a(M)$ excludes any of these that do not leave the machine in an accepting state. For example, any input sequence with two 1's in a row and the infinite sequence (1, 0, 1, 0, ...) are in $L(M)$ but not in $L_a(M)$.

Note that it is sometimes useful to consider **language containment** when referring to the language *accepted* by the state machine, rather than the language that gives all behaviors of the state machine.

Accepting states are also called **final states**, since for any behavior in $L_a(M)$, it is the last state of the machine. Accepting states are further explored in Exercise 2.

Regular Languages and Regular Expressions

A **language** is a set of sequences of values from some set called its **alphabet**. A language accepted by an FSM is called a **regular language**. A classic example of a language that is not regular has sequences of the form $0^n 1^n$, a sequence of n zeros followed by n ones. It is easy to see that no *finite* state machine can accept this language because the machine would have to count the zeros to ensure that the number of ones matches. And the number of zeros is not bounded. On the other hand, the input sequences accepted by the FSM in the box on page 380, which have the form $10101 \dots 01$, are regular.

A **regular expression** is a notation for describing regular languages. A central feature of regular expressions is the **Kleene star** (or **Kleene closure**), named after the American mathematician Stephen Kleene (who pronounced his name KLAY-nee). The notation V^* , where V is a set, means the set of all finite sequences of elements from V . For example, if $V = \{0, 1\}$, then V^* is a set that includes the **empty sequence** (often written λ), and every finite sequence of zeros and ones.

The Kleene star may be applied to sets of sequences. For example, if $A = \{00, 11\}$, then A^* is the set of all finite sequences where zeros and ones always appear in pairs. In the notation of regular expressions, this is written $(00|11)^*$, where the vertical bar means “or.” What is inside the parentheses defines the set A .

Regular expressions are sequences of symbols from an alphabet and sets of sequences. Suppose our alphabet is $A = \{a, b, \dots, z\}$, the set of lower-case characters. Then `grey` is a regular expression denoting a single sequence of four characters. The expression `grey|gray` denotes a set of two sequences. Parentheses can be used to group sequences or sets of sequences. For example, `(grey)|(gray)` and `gr(e|a)y` mean the same thing.

Regular expressions also provide convenience notations to make them more compact and readable. For example, the $+$ operator means “one or more,” in contrast to the Kleene star, which means “zero or more.” For example, `a+` specifies the sequences `a`, `aa`, `aaa`, etc.; it is the same as `a(a*)`. The $?$ operator specifies “zero or one.” For example, `colou?r` specifies a set with two sequences, `color` and `colour`; it is the same as `colo( $\lambda$ |u)r`, where λ denotes the empty sequence.

Regular expressions are commonly used in software systems for pattern matching. A typical implementation provides many more convenience notations than the ones illustrated here.

Example 14.6: Machines M_1 and M_2 in Figure 14.2 are language equivalent. Both machines produce output 1, 1, 0, 1, 1, 0, $\dots$, possibly interspersed with *absent* if the input is absent in some reactions.

Machine M_3 , however, has more behaviors. It can produce any output sequence that M_1 and M_2 can produce, but it can also produce other outputs given the same inputs. Thus, M_1 and M_2 are both language refinements of M_3 .

Language containment assures that an abstraction is **sound** with respect to **LTL** formulas about input and output sequences. That is, if A is a language refinement of B , then any LTL formula about inputs and outputs that holds for B also holds for A .

Example 14.7: Consider again the machines in Figure 14.2. M_3 might be a **specification**. For example, if we require that any two output values 0 have at

Probing Further: Omega Regular Languages

The **regular languages** discussed in the boxes on pages 380 and 381 contain only finite sequences. But embedded systems most commonly have infinite executions. To extend the idea of regular languages to infinite runs, we can use a **Büchi automaton**, named after Julius Richard Büchi, a Swiss logician and mathematician. A Büchi automaton is a possibly nondeterministic FSM that has one or more **accepting states**. The language accepted by the FSM is defined to be the set of behaviors that visit one or more of the accepting states **infinitely often**; in other words, these behaviors satisfy the **LTL** formula $\mathbf{GF}(s_1 \vee \dots \vee s_n)$, where $s_1, \dots, s_n$ are the accepting states. Such a language is called an **omega-regular language** or **ω -regular language**, a generalization of regular languages. The reason for using ω in the name is because ω is used to construct infinite sequences, as explained in the box on page 496.

As we will see in Chapter 15, many **model checking** questions can be expressed by giving a Büchi automaton and then checking to see whether the ω -regular language it defines contains any sequences.

least one intervening 1, then M_3 is a suitable specification of this requirement. This requirement can be written as an LTL formula as follows:

$$\mathbf{G}((y = 0) \Rightarrow \mathbf{X}((y \neq 0)\mathbf{U}(y = 1))).$$

If we prove that this property holds for M_3 , then we have implicitly proved that it also holds for M_1 and M_2 .

We will see in the next section that language containment is *not sound* with respect to LTL formulas that refer to states of the state machines. In fact, language containment does not require the state machines to have the same states, so an LTL formula that refers to the states of one machine may not even apply to the other machine. A sound abstraction that references states will require [simulation](#).

Language containment is sometimes called **trace containment**, but here the term “trace” refers only to the [observable trace](#), not to the [execution trace](#). As we will see next, things get much more subtle when considering execution traces.

14.4 Simulation

Two [nondeterministic FSMs](#) may be language equivalent but still have observable differences in behavior in some environments. Language equivalence merely states that given the same sequences of input [valuations](#), the two machines are *capable* of producing the same sequences of output valuations. However, as they execute, they make choices allowed by the nondeterminism. Without being able to see into the future, these choices could result in one of the machines getting into a state where it can no longer match the outputs of the other.

When faced with a nondeterministic choice, each machine is free to use any policy to make that choice. Assume that the machine cannot see into the future; that is, it cannot anticipate future inputs, and it cannot anticipate future choices that any other machine will make. For two machines to be equivalent, we will require that each machine be able to make choices that allow it to match the reaction of the other machine (producing the same outputs), and further allow it to continue to do such matching in the future. It turns out that language equivalence is not strong enough to ensure that this is possible.

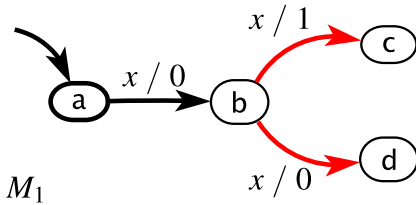
Example 14.8: Consider the two state machines in Figure 14.3. Suppose that M_2 is acceptable in some environment (every behavior it can exhibit in that environment is consistent with some specification or design intent). Is it safe for M_1 to replace M_2 ? The two machines are language equivalent. In all behaviors, the output is one of two finite strings, 01 or 00, for both machines. So it would seem that M_1 can replace M_2 . But this is not necessarily the case.

Suppose we compose each of the two machines with its own copy of the environment that finds M_2 acceptable. In the first reaction where x is *present*, M_1 has no choice but to take the transition to state b and produce the output $y = 0$. However, M_2 must choose between f and h. Whichever choice it makes, M_2 matches the output $y = 0$ of M_1 but enters a state where it is no longer able to always match the outputs of M_1 . If M_1 can observe the state of M_2 when making its choice, then in the second reaction where x is *present*, it can choose a transition that M_2 can *never* match. Such a policy for M_1 ensures that the behavior of M_1 , given the same inputs, is never the same as the behavior of M_2 . Hence, it is not safe to replace M_2 with M_1 .

On the other hand, if M_1 is acceptable in some environment, is it safe for M_2 to replace M_1 ? What it means for M_1 to be acceptable in the environment is that whatever decisions it makes are acceptable. Thus, in the second reaction where x is *present*, both outputs $y = 1$ and $y = 0$ are acceptable. In this second reaction, M_2 has no choice but to produce one or the other these outputs, and it will inevitably transition to a state where it continues to match the outputs of M_1 (henceforth forever *absent*). Hence it is safe for M_2 to replace M_1 .

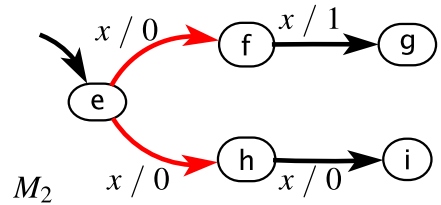
In the above example, we can think of the machines as maliciously trying to make M_1 look different from M_2 . Since they are free to use any policy to make choices, they are free to use policies that are contrary to our goal to replace M_2 with M_1 . Note that the machines do not need to know the future; it is sufficient to simply have good visibility of the present. The question that we address in this section is: under what circumstances can we assure that there is no policy for making nondeterministic choices that can make machine M_1 observably different from M_2 ? The answer is a stronger form of equivalence called [bisimulation](#) and a refinement relation called [simulation](#). We begin with the simulation relation.

input: x : pure
output: y : $\{0, 1\}$



(a)

input: x : pure
output: y : $\{0, 1\}$



(b)

Figure 14.3: Two state machines that are language equivalent but where M_2 does not simulate M_1 (M_1 does simulate M_2).

14.4.1 Simulation Relations

First, notice that the situation given in Example 14.8 is not symmetric. It is safe for M_2 to replace M_1 , but not the other way around. Hence, M_2 is a **refinement** of M_1 , in a sense that we will now establish. M_1 , on the other hand, is not a refinement of M_2 .

The particular kind of refinement we now consider is a **simulation refinement**. The following statements are all equivalent:

- M_2 is a simulation refinement of M_1 .
- M_1 simulates M_2 .
- M_1 is a simulation abstraction of M_2 .

Simulation is defined by a **matching game**. To determine whether M_1 simulates M_2 , we play a game where M_2 gets to move first in each round. The game starts with both machines in their initial states. M_2 moves first by reacting to an input valuation. If this involves a nondeterministic choice, then it is allowed to make any choice. Whatever it chooses, an output valuation results and M_2 's turn is over.

It is now M_1 's turn to move. It must react to the same input valuation that M_2 reacted to. If this involves a nondeterministic choice, then it must make a choice that matches the output valuation of M_2 . If there are multiple such choices, it must select one without knowledge of the future inputs or future moves of M_2 . Its strategy should be to choose one that enables it to continue to match M_2 , regardless of what future inputs arrive or future decisions M_2 makes.

Machine M_1 “wins” this matching game (M_1 simulates M_2) if it can always match the output symbol of machine M_2 for all possible input sequences. If in any reaction M_2 can produce an output symbol that M_1 cannot match, then M_1 does not simulate M_2 .

Example 14.9: In Figure 14.3, M_1 simulates M_2 but not vice versa. To see this, first play the game with M_2 moving first in each round. M_1 will always be able to match M_2 . Then play the game with M_1 moving first in each round. M_2 will not always be able to match M_1 . This is true even though the two machines are language equivalent.

Interestingly, if M_1 simulates M_2 , it is possible to compactly record all possible games over all possible inputs. Let S_1 be the states of M_1 and S_2 be the states of M_2 . Then a **simulation relation** $S \subseteq S_2 \times S_1$ is a set of pairs of states occupied by the two machines in each round of the game for all possible inputs. This set summarizes all possible plays of the game.

Example 14.10: In Figure 14.3,

$$S_1 = \{a, b, c, d\}$$

and

$$S_2 = \{e, f, g, h, i\}.$$

The simulation relation showing that M_1 simulates M_2 is

$$S = \{(e, a), (f, b), (h, b), (g, c), (i, d)\}$$

First notice that the pair (e, a) of initial states is in the relation, so the relation includes the state of the two machines in the first round. In the second round, M_2 may be in either f or h , and M_1 will be in b . These two possibilities are also accounted for. In the third round and beyond, M_2 will be in either g or i , and M_1 will be in c or d .

There is no simulation relation showing that M_2 simulates M_1 , because it does not.

A simulation relation is complete if it includes all possible plays of the game. It must therefore account for all **reachable states** of M_2 , the machine that moves first, because M_2 's moves are unconstrained. Since M_1 's moves are constrained by the need to match M_2 , it is not necessary to account for all of its reachable states.

14.4.2 Formal Model

Using the formal model of **nondeterministic FSMs** given in Section 3.5.1, we can formally define a **simulation relation**. Let

$$M_1 = (\text{States}_1, \text{Inputs}, \text{Outputs}, \text{possibleUpdates}_1, \text{initialState}_1),$$

and

$$M_2 = (\text{States}_2, \text{Inputs}, \text{Outputs}, \text{possibleUpdates}_2, \text{initialState}_2).$$

Assume the two machines are **type equivalent**. If either machine is deterministic, then its *possibleUpdates* function always returns a set with only one element in it. If M_1 simulates M_2 , the simulation relation is given as a subset of $\text{States}_2 \times \text{States}_1$. Note the ordering here; the machine that moves first in the game, M_2 , the one being simulated, is first in $\text{States}_2 \times \text{States}_1$.

To consider the reverse scenario, if M_2 simulates M_1 , then the relation is given as a subset of $\text{States}_1 \times \text{States}_2$. In this version of the game M_1 must move first.

We can state the “winning” strategy mathematically. We say that M_1 **simulates** M_2 if there is a subset $S \subseteq \text{States}_2 \times \text{States}_1$ such that

1. $(\text{initialState}_2, \text{initialState}_1) \in S$, and

2. If $(s_2, s_1) \in S$, then $\forall x \in \text{Inputs}$, and
 $\forall (s'_2, y_2) \in \text{possibleUpdates}_2(s_2, x)$,
 there is a $(s'_1, y_1) \in \text{possibleUpdates}_1(s_1, x)$ such that:
- (a) $(s'_2, s'_1) \in S$, and
 - (b) $y_2 = y_1$.

This set S , if it exists, is called the **simulation relation**. It establishes a correspondence between states in the two machines. If it does not exist, then M_1 does not simulate M_2 .

14.4.3 Transitivity

Simulation is **transitive**, meaning that if M_1 simulates M_2 and M_2 simulates M_3 , then M_1 simulates M_3 . In particular, if we are given simulation relations $S_{2,1} \subseteq \text{States}_2 \times \text{States}_1$ (M_1 simulates M_2) and $S_{3,2} \subseteq \text{States}_3 \times \text{States}_2$ (M_2 simulates M_3), then

$$S_{3,1} = \{(s_3, s_1) \in \text{States}_3 \times \text{States}_1 \mid \text{there exists } s_2 \in \text{States}_2 \text{ where } (s_3, s_2) \in S_{3,2} \text{ and } (s_2, s_1) \in S_{2,1}\}$$

Example 14.11: For the machines in Figure 14.2, it is easy to show that (c) simulates (b) and that (b) simulates (a). Specifically, the simulation relations are

$$S_{a,b} = \{(a, ad), (b, be), (c, cf), (d, ad), (e, be), (f, cf)\}.$$

and

$$S_{b,c} = \{(ad, ad), (be, bcef), (cf, bcef)\}.$$

By transitivity, we can conclude that (c) simulates (a), and that the simulation relation is

$$S_{a,c} = \{(a, ad), (b, bcef), (c, bcef), (d, ad), (e, bcef), (f, bcef)\},$$

which further supports the suggestive choices of state names.

14.4.4 Non-Uniqueness of Simulation Relations

When a machine M_1 simulates another machine M_2 , there may be more than one simulation relation.

Example 14.12: In Figure 14.4, it is easy to check that M_1 simulates M_2 . Note that M_1 is nondeterministic, and in two of its states it has two distinct ways of matching the moves of M_2 . It can arbitrarily choose from among these possibilities to match the moves. If from state b it always chooses to return to state a, then the simulation relation is

$$S_{2,1} = \{(ac, a), (bd, b)\}.$$

Otherwise, if from state c it always chooses to return to state b, then the simulation relation is

$$S_{2,1} = \{(ac, a), (bd, b), (ac, c)\}.$$

Otherwise, the simulation relation is

$$S_{2,1} = \{(ac, a), (bd, b), (ac, c), (bd, d)\}.$$

All three are valid simulation relations, so the simulation relation is not unique.

14.4.5 Simulation vs. Language Containment

As with all abstraction-refinement relations, simulation is typically used to relate a simpler specification M_1 to a more complicated realization M_2 . When M_1 simulates M_2 , then the language of M_1 contains the language of M_2 , but the guarantee is stronger than language containment. This fact is summarized in the following theorem.

Theorem 14.1. *Let M_1 simulate M_2 . Then*

$$L(M_2) \subseteq L(M_1).$$

input: x : pure
output: y : $\{0, 1\}$

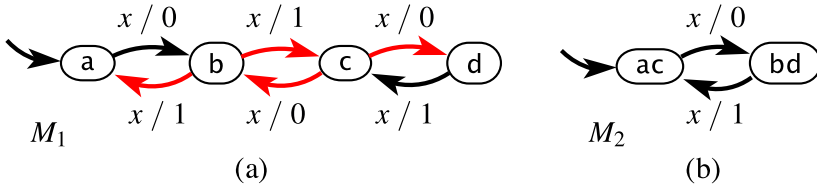


Figure 14.4: Two state machines that simulate each other, where there is more than one simulation relation.

Proof. This theorem is easy to prove. Consider a **behavior** $(x, y) \in L(M_2)$. We need to show that $(x, y) \in L(M_1)$.

Let the simulation relation be S . Find all possible **execution traces** for M_2

$$((x_0, s_0, y_0), (x_1, s_1, y_1), (x_2, s_2, y_2), \dots),$$

that result in behavior (x, y) . (If M_2 is deterministic, then there will be only one execution trace.) The simulation relation assures us that we can find an execution trace for M_1

$$((x_0, s'_0, y_0), (x_1, s'_1, y_1), (x_2, s'_2, y_2), \dots),$$

where $(s_i, s'_i) \in S$, such that given input valuation x_i , M_1 produces y_i . Thus, $(x, y) \in L(M_1)$.

□

One use of this theorem is to show that M_1 does not simulate M_2 by showing that M_2 has behaviors that M_1 does not have.

Example 14.13: For the examples in Figure 14.2, M_2 does not simulate M_3 . To see this, just note that the language of M_2 is a strict subset of the language of M_3 ,

$$L(M_2) \subset L(M_3),$$

meaning that M_3 has behaviors that M_2 does not have.

It is important to understand what the theorem says, and what it does not say. It does not say, for example, that if $L(M_2) \subseteq L(M_1)$ then M_1 simulates M_2 . In fact, this statement is not true, as we have already shown with the examples in Figure 14.3. These two machines have the same language. The two machines are observably different despite the fact that their input/output behaviors are the same.

Of course, if M_1 and M_2 are deterministic and M_1 simulates M_2 , then their languages are identical and M_2 simulates M_1 . Thus, the simulation relation differs from language containment only for nondeterministic FSMs.

14.5 Bisimulation

It is possible to have two machines M_1 and M_2 where M_1 simulates M_2 and M_2 simulates M_1 , and yet the machines are observably different. Note that by the theorem in the previous section, the languages of these two machines must be identical.

Example 14.14: Consider the two machines in Figure 14.5. These two machines simulate each other, with simulation relations as follows:

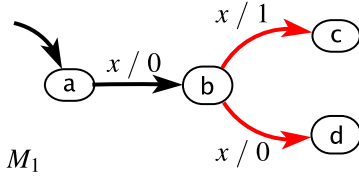
$$S_{2,1} = \{(e, a), (f, b), (h, b), (j, b), (g, c), (i, d), (k, c), (m, d)\}$$

(M_1 simulates M_2), and

$$S_{1,2} = \{(a, e), (b, j), (c, k), (d, m)\}$$

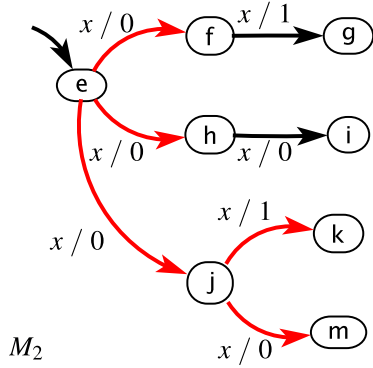
(M_2 simulates M_1). However, there is a situation in which the two machines will be observably different. In particular, suppose that the policies for making the

input: x : pure
output: y : $\{0, 1\}$



(a)

input: x : pure
output: y : $\{0, 1\}$



(b)

Figure 14.5: An example of two machines where M_1 simulates M_2 , and M_2 simulates M_1 , but they are not bisimilar.

nondeterministic choices for the two machines work as follows. In each reaction, they flip a coin to see which machine gets to move first. Given an input valuation, that machine makes a choice of move. The machine that moves second must be able to match all of its possible choices. In this case, the machines can end up in a state where one machine can no longer match all the possible moves of the other.

Specifically, suppose that in the first move M_2 gets to move first. It has three possible moves, and M_1 will have to match all three. Suppose it chooses to move to f or h . In the next round, if M_1 gets to move first, then M_2 can no longer match all of its possible moves.

Notice that this argument does not undermine the observation that these machines simulate each other. If in each round, M_2 always moves first, then M_1 will always be able to match its every move. Similarly, if in each round M_1 moves first, then M_2 can always match its every move (by always choosing to move to j in the first round). The observable difference arises from the ability to alternate which machines moves first.

To ensure that two machines are observably identical in all environments, we need a stronger equivalence relation called **bisimulation**. We say that M_1 is **bisimilar** to M_2 (or M_1 **bisimulates** M_2) if we can play the **matching game** modified so that in each round either machine can move first.

As in Section 14.4.2, we can use the formal model of **nondeterministic FSMs** to define a **bisimulation relation**. Let

$$\begin{aligned} M_1 &= (\text{States}_1, \text{Inputs}, \text{Outputs}, \text{possibleUpdates}_1, \text{initialState}_1), \text{ and} \\ M_2 &= (\text{States}_2, \text{Inputs}, \text{Outputs}, \text{possibleUpdates}_2, \text{initialState}_2). \end{aligned}$$

Assume the two machines are **type equivalent**. If either machine is deterministic, then its *possibleUpdates* function always returns a set with only one element in it. If M_1 bisimulates M_2 , the simulation relation is given as a subset of $\text{States}_2 \times \text{States}_1$. The ordering here is not important because if M_1 bisimulates M_2 , then M_2 bisimulates M_1 .

We say that M_1 **bisimulates** M_2 if there is a subset $S \subseteq \text{States}_2 \times \text{States}_1$ such that

1. $(\text{initialState}_2, \text{initialState}_1) \in S$, and
2. If $(s_2, s_1) \in S$, then $\forall x \in \text{Inputs}$, and
 $\forall (s'_2, y_2) \in \text{possibleUpdates}_2(s_2, x)$,
 there is a $(s'_1, y_1) \in \text{possibleUpdates}_1(s_1, x)$ such that:
 - (a) $(s'_2, s'_1) \in S$, and
 - (b) $y_2 = y_1$, and
3. If $(s_2, s_1) \in S$, then $\forall x \in \text{Inputs}$, and
 $\forall (s'_1, y_1) \in \text{possibleUpdates}_1(s_1, x)$,
 there is a $(s'_2, y_2) \in \text{possibleUpdates}_2(s_2, x)$ such that:
 - (a) $(s'_2, s'_1) \in S$, and
 - (b) $y_2 = y_1$.

This set S , if it exists, is called the **bisimulation relation**. It establishes a correspondence between states in the two machines. If it does not exist, then M_1 does not bisimulate M_2 .

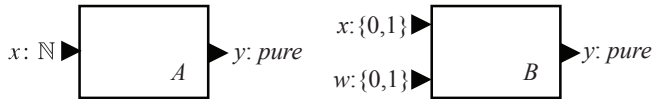
14.6 Summary

In this chapter, we have considered three increasingly strong abstraction-refinement relations for FSMs. These relations enable designers to determine when one design can safely replace another, or when one design correctly implements a specification. The first relation is type refinement, which considers only the existence of input and output ports and their data types. The second relation is language refinement, which considers the sequences of valuations of inputs and outputs. The third relation is simulation, which considers the state trajectories of the machines. In all three cases, we have provided both a refinement relation and an equivalence relation. The strongest equivalence relation is bisimulation, which ensures that two nondeterministic FSMs are indistinguishable from each other.

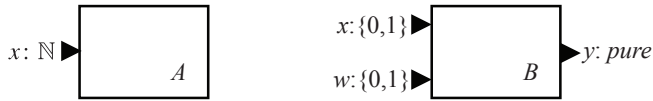
Exercises

1. In Figure 14.6 are four pairs of actors. For each pair, determine whether

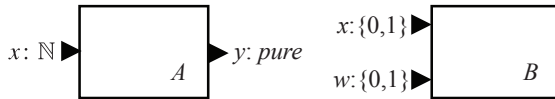
- A and B are type equivalent,
- A is a type refinement of B ,
- B is a type refinement of A , or
- none of the above.



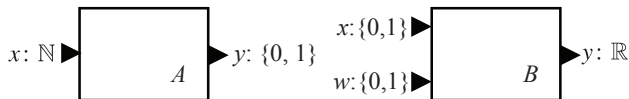
(a)



(b)



(c)



(d)

Figure 14.6: Four pairs of actors whose type refinement relationships are explored in Exercise 1.

2. In the box on page 380, a state machine M is given that accepts finite inputs x of the form (1) , $(1, 0, 1)$, $(1, 0, 1, 0, 1)$, etc.
 - (a) Write a **regular expression** that describes these inputs. You may ignore **stuttering** reactions.
 - (b) Describe the output sequences in $L_a(M)$ in words, and give a regular expression for those output sequences. You may again ignore stuttering reactions.
 - (c) Create a state machine that accepts *output* sequences of the form (1) , $(1, 0, 1)$, $(1, 0, 1, 0, 1)$, etc. (see box on page 380). Assume the input x is pure and that whenever the input is present, a present output is produced. Give a **deterministic** solution if there is one, or explain why there is no deterministic solution. What *input* sequences does your machine accept?
3. The state machine in Figure 14.7 has the property that it outputs at least one 1 between any two 0's. Construct a two-state nondeterministic state machine that simulates this one and preserves that property. Give the simulation relation. Are the machines bisimilar?
4. Consider the FSM in Figure 14.8, which recognizes an input code. The state machine in Figure 14.9 also recognizes the same code, but has more states than the one in Figure 14.8. Show that it is equivalent by giving a bisimulation relation with the machine in Figure 14.8.
5. Consider the state machine in Figure 14.10. Find a bisimilar state machine with only two states, and give the bisimulation relation.

inputs: $x: \{0, 1\}$
outputs: $y: \{0, 1\}$

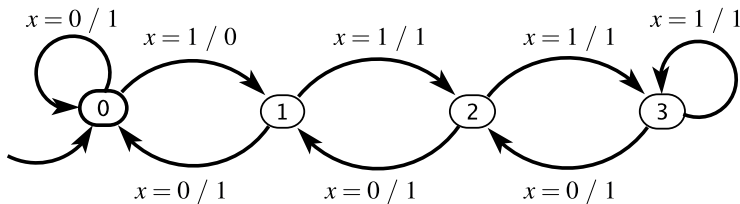


Figure 14.7: Machine that outputs at least one 1 between any two 0's.

input: $x: \{0,1\}$
output: *recognize*: pure

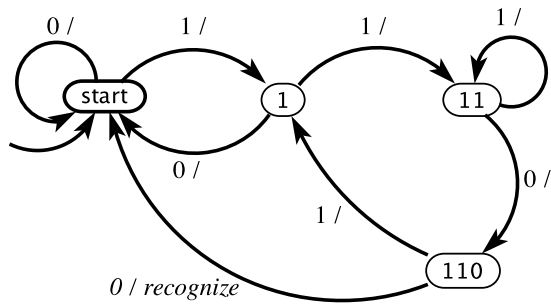


Figure 14.8: A machine that implements a code recognizer. It outputs *recognize* at the end of every input subsequence 1100; otherwise it outputs *absent*.

input: $x: \{0,1\}$
output: *recognize*: pure

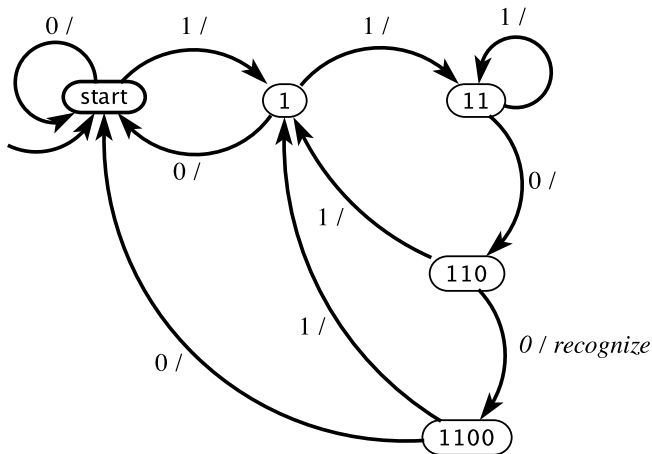


Figure 14.9: A machine that implements a recognizer for the same code as in Figure 14.8, but has more states.

6. You are told that state machine A has one input x , and one output y , both with type $\{1, 2\}$, and that it has states $\{a, b, c, d\}$. You are told nothing further. Do you have enough information to construct a state machine B that simulates A ? If so, give such a state machine, and the simulation relation.
7. Consider a state machine with a pure input x , and output y of type $\{0, 1\}$. Assume the states are

$$\text{States} = \{a, b, c, d, e, f\},$$

and the initial state is a . The *update* function is given by the following table (ignoring stuttering):

$(\text{currentState}, \text{input})$	$(\text{nextState}, \text{output})$
(a, x)	$(b, 1)$
(b, x)	$(c, 0)$
(c, x)	$(d, 0)$
(d, x)	$(e, 1)$
(e, x)	$(f, 0)$
(f, x)	$(a, 0)$

- (a) Draw the state transition diagram for this machine.
- (b) Ignoring stuttering, give all possible **behaviors** for this machine.
- (c) Find a state machine with three states that is bisimilar to this one. Draw that state machine, and give the bisimulation relation.
8. For each of the following questions, give a short answer and justification.

input: x : pure
output: y : $\{0, 1\}$

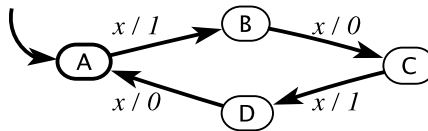


Figure 14.10: A machine that has more states than it needs.

- (a) TRUE or FALSE: Consider a state machine A that has one input x , and one output y , both with **type** $\{1, 2\}$ and a single state s , with two self loops labeled $true/1$ and $true/2$. Then for any state machine B which has exactly the same inputs and outputs (along with types), A simulates B .
- (b) TRUE or FALSE: Suppose that f is an arbitrary **LTL** formula that holds for state machine A , and that A simulates another state machine B . Then we can safely assert that f holds for B .
- (c) TRUE or FALSE: Suppose that A and B are two type-equivalent state machines, and that f is an LTL formula where the atomic propositions refer only to the inputs and outputs of A and B , not to their states. If the LTL formula f holds for state machine A , and A simulates state machine B , then f holds for B .